No Program

or Stop Writing Programs

Alexander Vityaz
Corezoid Inc., Dnipro, Ukraine
alexander.vityaz@gmail.com
corezoid.com · simulator.company
ORCID: 0009-0006-0489-7881
DOI: 10.5281/zenodo.23598371
Gemini Notebook

> "We will never run out of things to program as long as there is a single program around."
> — Alan Perlis, "Epigrams on Programming", no. 100, 1982.

Any serious change to a running system begins with an excavation. Before changing anything, the team has to work out how the system actually works: which exceptions have become rules, which assumptions made by people long gone still keep it from crashing, and where the code holds knowledge that nobody ever wrote down. We call this maintenance. In practice, it is archaeology: reconstructing a lost theory from the traces it left behind.

Peter Naur explained why changing a program first requires recovering the reasoning behind it. For Naur, the principal product of programming is the theory programmers build about how the program's execution solves a problem. Code and documentation express that theory without fully capturing it. Once the team that held the theory disperses, the program keeps running, but the ability to change it intelligently drops sharply. The codebase remains on the books as an asset, even as a growing part of it becomes accumulated uncertainty.

The industry has tried to solve this problem many times: new languages, frameworks, low-code, no-code, and now AI that writes code faster than a human. All of these speed up program creation. As long as the system's structure resides in separate programs and integrations, people must reconstruct it from code, diagrams, configuration, and the team's knowledge. Writing code faster does nothing, by itself, to reduce the excavations that lie ahead.

No Program starts with a question: how should the system work as a whole?

Before — The Program at the Center

We are used to thinking in terms of individual programs that together make up a company's IT system. In the classical approach, programming means describing algorithms and data structures in a language that helps a human express an intention while specifying exact rules for a computer to execute. The result is a program that becomes part of the IT system.

Niklaus Wirth's formula, "Algorithms + Data Structures = Programs," captures this approach.

- A distributed model. The system's structure has to be reconstructed from algorithms, data structures, configuration, and integrations. Even when an individual program is well organized, the relationships between programs need their own description and maintenance.

- Separate accounting. N programs mean N separate accounting systems within one IT system. Identities, states, and the outcomes of operations have to be reconciled across them.
- A fixed structure. Transactions change data and execution state while leaving the program's established structure intact. Changing that structure means returning to the code, settings, and dependent integrations.

Now — The System That Can Change

In No Program, we think in terms of the system as a whole: its purpose, actors, relationships, and rules of interaction. We define each part by the role it plays in the whole. The Actor Graph serves as an executable model from which programs and the connections between them are generated. When the tasks change, we transform the model and regenerate the affected programs and connections.

- Continuous interaction. A company keeps operating after any individual computation ends. The model must describe participants' states, messages, delays, obligations, and the next permitted actions.
- Model and regulation. The Conant–Ashby theorem links the simplest optimal regulator to a model of the system it regulates. No Program requires an explicit model sufficient for regulation. Noise suppression selects what matters for the goal; I examine its role in a separate paper on the factorization of minimal Good Regulators. Observability requires every accepted change to be reflected in actor state and causal history.
- AI and precise execution. A human sets the goal and constraints; AI proposes alternative system structures. A deterministic runtime executes the accepted rules. This makes it possible to search for solutions probabilistically while preserving the ability to verify actions, account for them, and assign responsibility for their outcomes.

Two Approaches to Development

Table 1. The Program and the System That Can Change

Dimension	Program	System That Can Change
Core metaphor	Cooking recipe / mathematical function	Cybernetic regulator / the organization's nervous system
Unit of design	Algorithms and data structures within programs	Actors, their states, and their relationships in the graph
Primary criterion	Program correctness and efficiency	Achieving the goal and maintaining plasticity: the system's ability to change while continuously accounting for those changes

The central problem of classical programming, then and now, is the fixed nature of code: the system's established structure is embedded in programs and the connections between them. Transactions change the data being processed, while the programs themselves remain static in this sense. When the system's structure changes—its actors, their relationships, or their rules of interaction—we have to return to the affected programs and integrations and rewrite the parts of the model embedded in them. First, though, we have to recover the meaning behind that code. As embedded decisions and dependencies accumulate, including dependencies between programs' separate accounting systems, this becomes increasingly difficult. That is how software turns into hardware.

Genetic Tools

Genetic tools therefore need to be built into the executable model of the entire system. They allow us to inherit and transform its structure, changing programs along with the connections between them. Whatever structure the system adopts must remain open to further change.

The idea has a long history. Back in 1958, Richard Friedberg investigated the automatic improvement of programs. Lisp (1958) and Prolog (1972) provided ways to treat programs as data: to generate and transform expressions and rules. In 1985, Nichael Cramer described constructing programs with genetic algorithms; an implementation of evolutionary search in Prolog followed in 1987. John Koza used Lisp and brought his work together in Genetic Programming (1992). Here programs form generations: they inherit fragments of their predecessors, undergo changes, and are selected based on their results.

But search costs and the difficulty of evaluation held back the adoption of this approach as a general method of development. Many programs had to be run against many examples, with success criteria defined in advance. Programs could grow without improving, and success on selected examples still offered no guarantee that they would work under new conditions.

Genetic search may produce a new program, but that program remains part of a larger system. If the solution becomes embedded in the program's code, its own accounting system, and API integrations, future changes still require coordinated edits to dependent programs. The more these dependencies accumulate, the harder it becomes to restructure the system. No Program moves genetic tools to the meta-level, where the structure of the entire system, its programs, and their connections can be inherited and transformed. AI proposes changes based on the goal, the live graph, and the transaction history. Candidate designs are tested on a copy of the graph and compared against a specified improvement criterion. Search costs are assessed separately.

Definition

No Program is a paradigm in which we think in terms of the system. We begin with the system's goal, its actors, relationships, and rules of interaction. The Actor Graph becomes an executable model of the whole system—its digital twin and metasystem. Programs and their connections are generated from this model.

Every element of the model is represented by an actor. Its relationships, states, messages, transactions, and rules determine how the system behaves. The model provides unified accounting for the entire system. Every transaction changes the graph—the system itself. Recording the transaction extends its causal history. Transactions can also create actors and transform their relationships and rules.

The generated programs and connections form a hierarchical network that mirrors the structure of the graph. Before approving a graph, we check its structure, permitted transitions, and invariants—the conditions the system must always satisfy. Alongside the approved model, we preserve its goal, constraints, and the explanation of why this structure was chosen. When requirements change, we first transform the model, then regenerate the affected programs and connections from the approved graph. We verify them before execution. We apply the verified and approved model change to the live system through transactions. Generated programs and connections can be deleted and regenerated, including for a different infrastructure. When generated programs are replaced, the graph retains actor identities, their accounts, and their transaction history.

Code is allowed inside actors and adapters, where the system meets operating systems, databases, protocols, and legacy infrastructure. Code generated from the graph remains derivative and replaceable. If a component contains handwritten code, that code must still be preserved and maintained; changes to it still require recovering the reasoning behind its internals. The graph defines the system's structure, including its connections to such components.

Every Step Up Was Metaprogramming

In the classical sense, metaprogramming means writing programs that generate and transform other programs. Assembly language let us express instructions and addresses symbolically. Compilers freed us from machine instructions; macros and generators freed us from repetitive code. At each step, humans moved up a level and stopped writing what the machine could now generate. When source code changes, the machine code produced by the compiler is generated again.

Lisp macros, C++ templates, reflection, and code generators all raised the level at which we work with programs. In No Program, the object of metaprogramming becomes the model of the entire system, including relationships between programs.

Changing an IT system's structure requires transforming both the programs and their connections. As long as those connections are embedded in API integration code, restructuring them requires reprogramming the dependent programs.

Each program has its own accounting system: its own identities, states, and rules for recording operations. N programs mean N accounting systems within one IT system. Each creates its own history. For the programs to operate as one system, integrations have to match identities, propagate changes, and reconcile accounting results. The rules for that reconciliation become embedded in integration code. Change the structure of one accounting system, and you also have to revisit the rules that reconcile it with the others. The more dependencies of this kind accumulate, the harder the whole system becomes to restructure. This is one reason software hardens.

Models have served as a foundation for development before: CASE, Model-Driven Architecture, and executable UML. MDA provided for modeling the relationships between applications and generating the corresponding implementations. ActionWorkflow represented a business process as a network of commitment cycles between participants. WfMC and BPMN standards advanced executable process descriptions. These approaches showed that a model could define the behavior of individual programs as well as their interactions. In No Program, the model must preserve participant identity and accounting continuity as the entire system changes.

Code, Low-Code, No-Code, and No Program

A real IT system is always hybrid: its components are built in different ways. No Program can use code, low-code, and no-code for this purpose. Table 2 compares the development of separate programs with their own accounting systems against working with a single model of the entire system.

Table 2. Developing Separate Programs vs. Working with a Model of the Entire System

	Code	**Low-code**	**No-code**	**No Program**
What a person works with	Program text	A program assembled from visual blocks and code snippets	Ready-made blocks and platform settings	An Actor Graph from which programs and their connections are derived
What gets built	A program as part of the system	A program as part of the system	A program as part of the system	An executable model of the entire system, serving as its digital twin and metasystem
How accounting works	Program-specific accounting	Program-specific accounting	Program-specific accounting	Unified accounting for the entire system in the Actor Graph

	Code	Low-code	No-code	No Program
How the structure changes	Editing code and dependent integrations	Editing blocks and code snippets	Reconfiguring within the platform's capabilities	Changing and verifying the graph; regenerating and verifying the resulting programs and connections
Genetic tools	Inheriting and transforming programs and code fragments	Reusing and changing application models and blocks	Copying and reconfiguring application blocks	Inheriting and transforming the graph, programs, and connections. The approved version retains the means for further change.
Strength	Full control, unrestricted expressiveness, mature tooling	Rapid development of standard applications, with the option to add custom code	Accessible to non-programmers; quick results in a narrow domain	An exact executable model of the entire system: programs and connections can change together while actor identities and accounting continuity are preserved
What a transaction changes	Data and execution state. The program's established structure remains intact.	Data and execution state. The program's established structure remains intact.	Data and execution state. The program's established structure remains intact.	The graph itself—the state of the system. Recording the transaction extends causal history. A transaction can also change actors, connections, and rules.

What the Graph Is Made Of

An actor here is a broader concept than an actor in the classical model of concurrent computation. It is a unit of the reality being modeled: a person, organization, document, device, contract, process, or software component. That does not mean every noun in a domain description becomes an actor. We represent something as an actor when the validity or outcome of a transaction, its accounting, rights, or risk depend on it.

The actor contract is minimal: a global identifier, accounts, events, and a process. The identifier belongs to the actor itself and persists across changes of runtime, application, and implementation. Accounts record money, statuses, obligations, rights, limits, and versions; every value has a defined type. Events preserve causal history: how the current state came about. A process defines transition rules and may itself be an ordinary process, another actor, or a nested graph.

An edge is an actor too. A relationship cannot be reduced to a field in a record if doing so changes the outcome, accounting, rights, or responsibility. Such a relationship must be an actor with the same contract.

Transactions create the graph. Every subsequent transaction changes it. A message recognized by the system becomes a transaction: it connects identities, changes accounts, generates events, and invokes a process. Changes to accounts use double-entry accounting. Reversals and compensations are

recorded as new transactions; the past is never rewritten. This is how accounting creates the system's causal history. The graph preserves that history alongside its current state.

The accounts of different actors form a single accounting system for the whole system. Programs address those actors by their global identifiers, and transactions link changes across their accounts. This allows programs and their connections to be transformed while preserving actor identities and continuity of accounting.

A Hierarchical Network of Programs

The Actor Graph generates a network of programs and connections structured like the system itself. Every actor executes its process, and a process can unfold into a nested graph with its own actors and programs.

An actor retains the same identifier in every subgraph in which it participates. Shared actors connect subgraphs into a network. An entire graph can be represented as a single actor and then expanded again. Its internal actors retain their identities and transactional memory throughout.

Variants for simulation are built from a copy of the approved graph. New actors and connections receive their own identities. The copy preserves the history up to the moment it was created, along with mappings to the original actors. Subsequent transactions belong to the variant and do not change accounts in the live system.

Changing the graph's structure may affect both programs and their connections. After verifying and approving the modified model, we regenerate the relevant programs and connections. We test them together with other components affected by the change. If modifying one program requires changes to others, the graph shows which ones.

Initial development accounts for only part of a system's cost. For years afterward, programs and their connections have to be changed. No Program should reduce the cost of those changes: dependencies are explicit, affected components are verified before execution, and approved changes and their outcomes are preserved in the transaction history.

A graph can contain and generate other graphs. This is metaprogramming with no top floor.

AI as the Metaprogrammer's Interface

This essay continues my paper Beyond Programming Languages. It connects the move to an executable model with unified accounting and genetic tools that preserve the entire system's capacity to change.

AI makes it possible to separate two functions of a programming language: expressing human intent and specifying precise execution rules. Working with AI, a person describes a goal in ordinary language and supplies examples, documents, tables, and records of real operations. AI proposes a system model—actors, their connections, states, and transitions—and looks for contradictions.

But an understandable description still falls short of precise rules. "Approve loans quickly for good customers" makes sense to a human, yet it does not define a system. Who counts as a good customer? What does quickly mean? Who has the authority to decide? Can the decision be reversed? A language model proposes an interpretation, but cannot prove that it reflects the system owner's intent.

In No Program, AI makes its proposed changes to the graph explicit. A deterministic runtime executes the accepted rules.

Programming with AI: intent → AI → program → execution.

No Program: intent → AI → Actor Graph → generated programs and connections → execution.

Precise execution does not guarantee that the system model is correct. The model must be verified and tested in simulation before approval. Generated programs and connections are checked against their dependencies before execution. Design, simulation, and execution use different tools.

Genetic tools operate at the meta-level, on the model of the entire system. Inheritance makes it possible to build a new structure from a working graph. Variability comes from operations on that model: combining parts of graphs, creating actors, and changing their processes, connections, and rules. An approved variant remains a graph that can be inherited and transformed again. The means of making further changes remain part of the model.

AI proposes changes aimed at achieving the goal. Verified variants are compared in simulation against a predefined improvement criterion, and a suitable variant is selected. Accounting for the outcomes of real operations provides feedback for subsequent changes.

This Text Was Published Through a Graph

Corezoid.com publications and inquiries are represented in a single twin graph in Simulator.Company and belong to a unified accounting system. Every page, news item, and case study is an actor in that graph, with its own identifier, revisions, and publication status. A request submitted through a website form becomes a transaction in the same graph, and its state changes according to the rules defined in the system-wide model. There is no separate content management system or separate system for handling inquiries. Publication follows a sequence: AI Shift, an AI development system, creates a draft through MCP; the graph checks whether publication is permitted under the editorial rules in force; the website then displays the published page. That is how the site operates. It would be strange to have to write a separate program just to publish an essay about No Program.

From Programs to Systems

Programming languages accelerated development and freed people from having to think in machine instructions. Machine code will remain, because machines execute it. The program as a product that a human writes and keeps in their head for years may disappear, because humans were its primary audience. That is our answer to Perlis: in No Program, a human thinks in terms of the system, while programs and their connections are derived from the model of the whole. Genetic tools preserve that system's ability to change.

Stop writing programs. Build systems.

References

1. Perlis, Alan J. Special Feature: Epigrams on Programming. ACM SIGPLAN Notices. 1982. Vol. 17, no. 9. Pp. 7–13. Epigram no. 100. DOI: 10.1145/947955.1083808.
2. Wirth, Niklaus. Algorithms + Data Structures = Programs. Englewood Cliffs, NJ: Prentice-Hall, 1976. ISBN 0-13-022418-9. Author's bibliography.
3. Conant, Roger C.; Ashby, W. Ross. Every Good Regulator of a System Must Be a Model of That System. International Journal of Systems Science. 1970. Vol. 1, no. 2. Pp. 89–97. DOI: 10.1080/00207727008920220.
4. Vityaz, Alexander. On the Necessity of Noise Suppression for Minimal Good Regulators: Factorization Theorems and a Closure Conjecture. Preprint, January 2026. DOI: 10.13140/RG.2.2.33143.07843.
5. Friedberg, R. M. A Learning Machine: Part I. IBM Journal of Research and Development. 1958. Vol. 2, no. 1. Pp. 2–13. DOI: 10.1147/rd.21.0002.
6. McCarthy, John. History of Lisp. Artificial Intelligence Laboratory, Stanford University. Author's version dated February 12, 1979.

7. Colmerauer, Alain; Roussel, Philippe. The Birth of Prolog. Author's manuscript, November 1992.
8. Cramer, Nichael Lynn. A Representation for the Adaptive Generation of Simple Sequential Programs. In: Grefenstette, John J. (ed.). Proceedings of the First International Conference on Genetic Algorithms and Their Applications. 1985. Pp. 183–187.
9. Dickmanns, Dirk; Schmidhuber, Jürgen; Winklhofer, Andreas. Der genetische Algorithmus – eine Implementierung in Prolog. Fortgeschrittenenpraktikum, Institut für Informatik, Lehrstuhl Prof. Radig, Technische Universität München, 1987.
10. Koza, John R. Genetic Programming: On the Programming of Computers by Means of Natural Selection. The MIT Press, 1992. ISBN 978-0-262-11170-6.
11. Poli, Riccardo; Langdon, William B.; McPhee, Nicholas Freitag. A Field Guide to Genetic Programming. Lulu.com, 2008. ISBN 978-1-4092-0073-4. With contributions from John R. Koza. See § 3.3, p. 24 (PDF page 38), and § 11.3, p. 101 (PDF page 115).
12. Naur, Peter. Programming as Theory Building. Microprocessing and Microprogramming. 1985. Vol. 15, no. 5. Pp. 253–261. DOI: 10.1016/0165-6074(85)90032-8.
13. Object Management Group. Model Driven Architecture (MDA): MDA Guide rev. 2.0. OMG Document ormsc/2014-06-01. June 2014.
14. Medina-Mora, Raul; Winograd, Terry; Flores, Rodrigo; Flores, Fernando. The Action Workflow Approach to Workflow Management Technology. Proceedings of the 1992 ACM Conference on Computer-Supported Cooperative Work (CSCW '92). New York: ACM, 1992. Pp. 281–288. DOI: 10.1145/143457.143530.
15. Hollingsworth, David. The Workflow Reference Model. Workflow Management Coalition. Document TC00-1003. Issue 1.1. January 19, 1995.
16. Object Management Group. Business Process Model and Notation (BPMN). Version 2.0.2. OMG Document formal/2013-12-09. January 2014.
17. Mendix. Best Practices for Adaptability. Documentation. Updated July 7, 2026. Accessed October 19, 2026.
18. Bubble. Top and context menu options. Documentation. Accessed October 19, 2026.
19. Vityaz, Alexander. Beyond Programming Languages. Version v1, July 2026. DOI: 10.5281/zenodo.21458098.